



umwl-tui — from an Illumio flow export to loaded policy objects

Exporting from the PCE, anonymizing, analysing with Claude, reviewing the proposals and loading unmanaged workloads and IP lists with reconciliation by IP and a full audit trail per run

Eric Roscher – Illumio LATAM Pre-Sales Engineering

September 2026

Working guide prepared by Eric Roscher – Illumio LATAM Pre-Sales Engineering — not an official Illumio, Inc. publication.

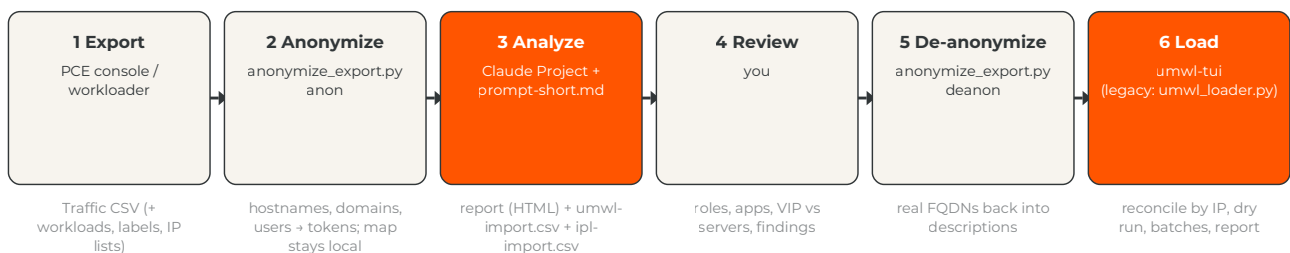
■ Contents

01	The workflow at a glance	3
02	Requirements and one working folder per Illumio account + PCE	5
03	Initialization	8
04	Exporting from the PCE	11
05	Anonymization	15
06	Running the analysis with Claude	17
07	Policy objects and naming convention	19
08	umwl-tui step by step	22
09	umwl-tui and workloader	26
10	Iterating: v2 exports	29
11	Troubleshooting and checklist	29
12	Appendix A — Export columns	32
13	Appendix B — Files in the repository	33

■ The workflow at a glance

This guide covers the whole path from a PCE traffic export to policy objects loaded in that same PCE: exporting the right data, pseudonymizing it, running the flow analysis with Claude, reviewing what comes back, restoring the real names and loading the unmanaged workloads and IP lists. The six steps below are the skeleton of the document; each one has its own section.

THE SIX STEPS: EXPORT, ANONYMIZE, ANALYZE, REVIEW, DE-ANONYMIZE, LOAD



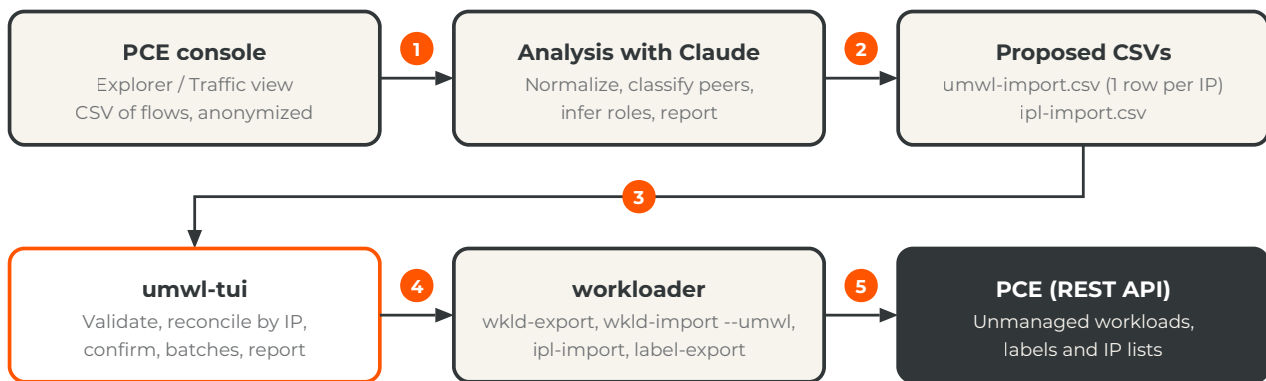
Never leaves the working folder: pce.yaml (API key), anon-map.json (the pseudonym map), runs/ (inventories and logs)

1. Traffic export from the Explore > Traffic view (mandatory) plus the workloads, labels, label types and IP lists already in the PCE (recommended). Section 4.
2. `anonymize_export.py anon` replaces hostnames, domains, the customer's name and personal usernames with consistent tokens; the map stays in the working folder. Section 5.
3. A Claude Project with `docs/prompts/context.md` as instructions and the short prompt per conversation returns the report and the two CSVs. Section 6.
4. A person checks roles, applications, the VIP versus front-end distinction and the findings; corrections go back to the same conversation. Section 6.
5. `anonymize_export.py deanon` puts the real FQDNs back into the descriptions of the proposed CSVs. Section 5.
6. `umwl-tui` reconciles every row by IP against the PCE, dry-runs, asks, writes in batches and leaves the report. Sections 8 and 9.

What the product is

`umwl-tui` is a full-screen terminal application written in Go. It takes the unmanaged workloads (UMWL) and IP lists proposed by the analysis, already in CSV format, and loads them into the PCE through `workloader`, the open-source command-line tool for the PCE REST API published on GitHub as `brian1917/workloader`¹. What the application adds on top of a bulk import is exactly what `workloader` does not do by itself: validate the CSV, reconcile every IP against the live inventory of the PCE, ask a person before each decision that matters, dry-run, write in batches with retry and skip, verify, and persist everything it decided or observed in `runs/<timestamp>/`. It is the primary method described in this guide; the earlier plain-terminal Python implementation, `legacy/umwl_loader.py`, is kept in the repository only as a fallback and follows the same ten steps.

END-TO-END FLOW: FROM THE TRAFFIC EXPORT TO OBJECTS IN THE PCE



1. Export of the Explore > Traffic view with the time window and filters of the proof of concept, pseudonymized before it leaves the working folder; opened as text, never saved from Excel.
2. Normalization (mangled IPs, dates, truncation), the unmanaged workload versus IP list criterion, evidence per port, process and volume, label model and findings.
3. The analysis delivers the CSVs in the `wkld-import` and `ipl-import` contract, with the customer's own label values and the evidence comment in `description`; `deanon` restores the real names before loading.
4. `umwl-tui` calls workloader subcommands: read-only first (inventory), then a dry run and, after confirmation, batched writes.
5. workloader talks to the PCE REST API using the `pce.yaml` of the working folder.

ONE WORKING FOLDER PER ILLUMIO ACCOUNT + PCE

workloader reads `./pce.yaml` from the folder it runs in (unless `--config-file` or `ILLUMIO_CONFIG` is set)², and `umwl-tui` writes `runs/<timestamp>/` in that same folder. The isolation unit is each **Illumio account or organization + PCE** combination (each distinct `pce.yaml` profile: FQDN, port, org id, API key); sharing a folder between two of them risks importing into the wrong tenant. Structure in section 2; commands in section 3.

SOURCES — OFFICIAL DOCUMENTATION

1. workloader (brian1917) — README, releases v12.1.9 — github.com/brian1917/workloader
2. workloader — `cmd/root.go`: configuration file lookup order (`--config-file`, `ILLUMIO_CONFIG`, `./pce.yaml`) and persistent flags — github.com/brian1917/workloader/blob/master/cmd/root.go

■ Requirements and one working folder per Illumio account + PCE

`umwl-tui` is a single static binary with no runtime dependency other than the `workloader` binary in the working folder or on the PATH. Python 3.9 or later is only needed for `anonymize_export.py` (standard library, no `pip install`) and for the legacy loader. What the workflow does need is an identity with write permission on the PCE and HTTPS access to its API.

REQUIREMENTS AND FOLDER STRUCTURE: ONE PER ACCOUNT + PCE

Mac (Apple Silicon or Intel)

`umwl-tui`

Go binary · release or make build

`workloader v12.x`

Intel binary + Rosetta, or go build

`~/illumio/customer-a/scp57-org12/`

`umwl-tui`

`workloader`

`pce.yaml`

`kit/`

`customer-a-umwl-import.csv`

`customer-a-ipl-import.csv`

`anon-map.json`

`runs/20260903-101500/`

the Go app (GitHub release)

binary; `umwl-tui` downloads it

ONLY this account + PCE

clone: `anonymize_export.py`, ...
after `deanon`

pseudonym map; never leaves

inventory, logs, report

`~/illumio/customer-a/onprem-prod/`

`umwl-tui workloader pce.yaml ...`

same account, another PCE

`~/illumio/customer-b/<pce-or-org>/`

`umwl-tui workloader pce.yaml ...`

another account, same layout

1

SaaS PCE scp57

org 12 · HTTPS 443
API key with a write role

2

On-prem prod PCE

same account, other `pce.yaml`

3

customer-b PCE

other account, other folder

1. A SaaS PCE (one FQDN) can host several organizations; each org id is a separate tenant with its own folder. `workloader` authenticates with the `pce.yaml` of the current folder; the preflight shows the `pce-list` result before going on.
2. The same customer with another PCE (on-prem, DR, another region) is another folder with its own `pce.yaml`; the binaries can be shared through a symlink.
3. Another account: same layout, never the same `pce.yaml`, never the same `anon-map.json`.

The isolation unit: Illumio account or organization + PCE

A SaaS PCE FQDN can host several organizations (the `org id` is the number that appears in the console URL, `.../orgs/<id>/...`), and a single customer can have several PCEs: SaaS and on-prem, production and DR, regions. That is why the isolation unit is neither "the customer" nor "the PCE" but each **account or organization + PCE** combination, that is, each distinct `pce.yaml` profile (FQDN, port, org id and API key): the same customer with two orgs on the same PCE is two folders.

workloader supports several profiles in a single `pce.yaml`, with a `default_pce_name` and the `--pce <name>` flag to pick one per command⁵. The kit deliberately does not rely on that: a forgotten `--pce` or a wrong default would load the objects into the wrong tenant. Suggested naming: `~/illumio/<account>/<pce-or-org>/`, for example `~/illumio/customer-a/scp57-org12/`.

COMPONENT	DETAIL
Operating system	macOS on Apple Silicon or Intel (tested), or Linux. The terminal must be at least 80×24; the interface is in English.
umwl-tui	Prebuilt binaries attached to the GitHub releases of the kit (<code>umwl-tui-darwin-arm64</code> , <code>umwl-tui-darwin-amd64</code> , <code>umwl-tui-linux-amd64</code> , <code>umwl-tui-linux-arm64</code> , plus <code>SHA256SUMS</code>). On a Mac: <code>chmod +x</code> and <code>xattr -d com.apple.quarantine umwl-tui</code> . Alternative: Go 1.24+ and <code>make build</code> from a clone.
workloader v12.x	Preferably built from source by the preflight of <code>umwl-tui</code> (key <code>b</code>): it clones <code>brian1917/workloader</code> ¹ at the latest release tag into <code>workloader-src/</code> and runs <code>go build</code> , which gives a native binary for the machine (arm64 on Apple Silicon) with a reproducible version. That needs Go 1.24+ and <code>git</code> (<code>brew install go</code> ; <code>git</code> comes with <code>xcode-select --install</code>). The reason to prefer it: the upstream macOS release (<code>mac-v12.1.9.zip</code>) is an Intel-only binary that runs under Rosetta 2. The release download stays as the fallback (key <code>d</code>) for machines without Go. A binary that already exists anywhere on disk can be used as is (key <code>w</code> , remembered in <code>umwl-tui.json</code>).
Working folder	One per account or organization + PCE: it holds <code>umwl-tui</code> , <code>workloader</code> , <code>pce.yaml</code> , the exports and the CSVs, <code>anon-map.json</code> and <code>runs/</code> . A clone of the repository in <code>kit/</code> is optional (it provides <code>anonymize_export.py</code> , the prompts and the example CSVs).
PCE API key	User menu → My API Keys → Add ; save the <i>Authentication Username</i> (<code>api_...</code>) and the <i>Secret</i> , shown only once. It inherits the user's role: creating workloads, labels and IP lists requires Global Organization Owner or Global Administrator ^{2,4} . On SaaS the <code>org id</code> is the number in the URL.
Network	HTTPS from the workstation to the PCE FQDN (443 on SaaS, typically 8443 on-prem). workloader needs nothing else ³ . Downloading workloader from the preflight needs HTTPS to github.com.

PCE.YAML CONTAINS THE API KEY; ANON-MAP.JSON CONTAINS THE REAL NAMES

Do not copy either file between folders, attach it to a conversation or push it to a repository. The kit's `.gitignore` excludes both, together with the binaries, the logs, `runs/` and the `*.anon.csv` / `*.real.csv` copies. workloader creates `pce.yaml`; restrict it with `chmod 600 pce.yaml`.

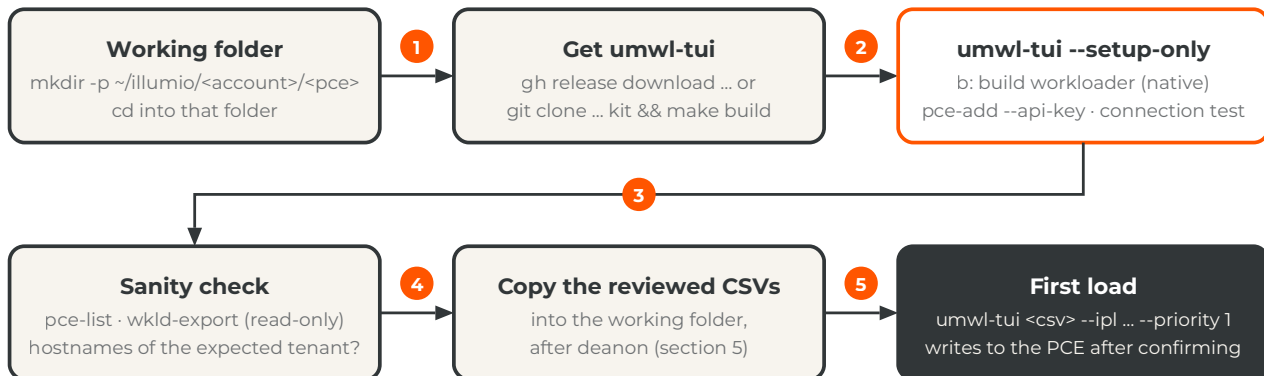
SOURCES — OFFICIAL DOCUMENTATION

1. workloader v12.1.9 — release assets (mac-v12.1.9.zip, linux_amd64, linux_arm, windows) — github.com/brian1917/workloader/releases/tag/v12.1.9
2. Illumio Core 24.2 — REST API — API Keys (My API Keys, the key inherits the user's role) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/REST-APIs/out/en/rest-apis/authentication-and-api-user-permissions/api-keys.html
3. workloader — README: "The only requirements for workloader are the ability to run the executable and connect to the PCE over HTTPS" — github.com/brian1917/workloader
4. Illumio Core 25.4 — Admin — Roles, Scopes, and Granted Access — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Admin/out/en/pce-administration/role-based-access-control/roles-scopes-and-granted-access.html
5. workloader — README and cmd/pcemgmt/addpce.go: several PCEs in one pce.yaml, default_pce_name and the --pce flag — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go

Initialization

These steps are done once per account + PCE combination. They leave a folder with the `umwl-tui` binary, the workloader binary and a tested `pce.yaml`; after that, every load is a single command. The kit repository is private: a GitHub session (`gh auth login` or an SSH key loaded into the account) is needed to download the release or to clone.

INITIALIZATION: FROM AN EMPTY FOLDER TO THE FIRST LOAD



1. The folder is the isolation unit; everything that follows runs with that folder as the current directory.
2. The binary comes from the GitHub release (checksum in `SHA256SUMS`) or from `make build` in a clone; the clone in `kit/` also provides `anonymize_export.py` and the prompts.
3. `--setup-only` runs the preflight only: it locates `workloader` or builds it from source (`b` , native; `d` downloads the release as a fallback), opens the `pce-add --api-key` form when `pce.yaml` has no PCE, and tests the connection with a read-only `label-dimension-export` .
4. Before writing anything, a read-only export proves that the API key works and that the inventory belongs to the expected tenant.
5. The CSVs that come back from the analysis, reviewed and with the real names restored, go into the working folder named after the customer.
6. `umwl-tui` walks through steps 0 to 10 of section 8; it writes to the PCE only after the "Write to the PCE?" confirmation that follows the dry run.

Commands

TERMINAL · A. WORKING FOLDER FOR THIS ACCOUNT + PCE

```
mkdir -p ~/illumio/customer-a/scp57-org12 && cd ~/illumio/customer-a/scp57-org12
```


TERMINAL · B. UMWL-TUI FROM THE GITHUB RELEASE (PRIVATE REPOSITORY: GH AUTH LOGIN FIRST)

```
gh release download --repo roschereric/illumio-workloader-import-kit \
  --pattern 'umwl-tui-darwin-arm64' --pattern SHA256SUMS
shasum -a 256 -c SHA256SUMS --ignore-missing
mv umwl-tui-darwin-arm64 umwl-tui && chmod +x umwl-tui && xattr -d com.apple.quarantine umwl-tui
# Intel Mac: umwl-tui-darwin-amd64 · Linux: umwl-tui-linux-amd64 / umwl-tui-linux-arm64

# alternative: build it (Go 1.24+); the clone also gives you anonymize_export.py and docs/prompts/
git clone https://github.com/roschereric/illumio-workloader-import-kit.git kit
(cd kit && make build) && cp kit/umwl-tui .
```

TERMINAL · C. WORKLOADER AND PCE.YAML WITH THE PREFLIGHT OF UMWL-TUI

```
./umwl-tui --setup-only
```

The preflight shows three checks: *workloader binary*, *pce.yaml* and *PCE connection*, and an *Environment* block with the folder, the OS/CPU and the toolchain found (`go` , `git` , `brew`). It looks for workloader in this order: `--workloader <path>` , the path saved in `./umwl-tui.json` or `~/.config/umwl-tui/config.json` , `./workloader` , the PATH. The check row prints the version and the CPU architecture of what it found — `arm64` , `native` , or a warning when an Intel binary would run under Rosetta 2 on an Apple Silicon Mac. Three keys install or select the binary:

KEY	PATH	WHAT HAPPENS
b	Build from source (recommended)	Confirmation modal, then <code>git clone --depth 1 --branch <latest tag> https://github.com/brian1917/workloader-workloader-src</code> and <code>CGO_ENABLED=0 go build -trimpath -ldflags "... utils.Version=\$(cat version) ..." -o ./workloader .</code> — the same flags as workloader's own release workflow, so <code>workloader version</code> reports the tag. The clone gets a different name than the binary (<code>workloader-src</code>) so <code>./workloader</code> can sit next to it. If Go is missing and Homebrew is present, the modal offers <code>brew install go</code> and then builds; without Homebrew it prints the manual commands (next block). Output streams into the log pane; the first build downloads workloader's Go modules (one to three minutes), later builds take seconds.
d	Download the release (fallback)	Confirmation modal, then the pre-built release for the current OS (<code>curl -fsSL</code> from github.com, <code>unzip</code> , <code>chmod</code> , on macOS <code>xattr -d com.apple.quarantine</code>). On darwin/arm64 the modal states that this binary is Intel-only and will run under Rosetta 2.
w	Use a workloader elsewhere	Opens the file chooser (type a path on top with <code>tab</code> ; <code>~</code> works), checks the execute bit and reads the version, then asks where to remember the choice: <code>l</code> this folder (<code>./umwl-tui.json</code>) or <code>u</code> this user, every folder (<code>~/.config/umwl-tui/config.json</code>). Both files hold only the path, no secrets; the local one wins; <code>--workloader</code> on the command line overrides both for one run. This is how one binary built once in <code>~/tools/</code> serves every working folder without editing the PATH.

It then runs `pce-list` ; when `pce.yaml` does not exist or workloader answers "no pce configured", the key `p` opens a form with short name, FQDN, port, API user, API secret (masked), org id and the TLS-verification switch, and runs `pce-add --api-key ...` with those values². The secret is passed as an argument to workloader, never through a shell, and is masked in the log pane and in the report. The key `t` repeats the connection test (a read-only `label-dimension-export`) and `r` re-runs the three checks. With all three green the screen says "Setup complete."; `q` quits.

TERMINAL · C'. THE SAME BUILD BY HAND (WHEN B CANNOT RUN, OR TO BUILD ONCE IN A SHARED FOLDER)

```
brew install go # once per Mac (Go 1.24+); git: xcode-select --install
git clone --depth 1 --branch v12.1.9 https://github.com/brian1917/workloader workloader-src
cd workloader-src
CGO_ENABLED=0 go build -trimpath -o ../workloader \
  -ldflags "-s -w -X github.com/brian1917/workloader/utils.Version=$(cat version) \
    -X github.com/brian1917/workloader/utils.Commit=$(git rev-list -1 HEAD)" .
cd .. && ./workloader version && file ./workloader # tag, and "Mach-O 64-bit executable arm64"
# tags: git ls-remote --tags https://github.com/brian1917/workloader | tail
# update later: git -C workloader-src fetch --tags && git -C workloader-src checkout v12.2.0, build again
# the result is one static file: cp workloader ~/tools/workloader, then w in each working folder
```

TERMINAL · D. SANITY CHECK (READ-ONLY)

```
./workloader pce-list
./workloader wkld-export --output-file sanity.csv
# look at the hostnames in sanity.csv: is this the inventory of the expected tenant?
```

RESULTING LAYOUT

```
~/illumio/customer-a/scp57-org12/
umwl-tui # the Go application (release binary or make build)
workloader # native binary built by the preflight (b) – or a path saved in umwl-tui.json
workloader-src/ # clone of brian1917/workloader used for the build (gitignored)
umwl-tui.json # optional: path to a workloader binary elsewhere (no secrets)
pce.yaml # ONLY this account + PCE; created by pce-add; never copy it
kit/ # optional clone: anonymize_export.py, docs/prompts/, examples/
TrafficData 9_2_2026.csv # exports from the PCE (section 4)
anon-map.json # pseudonym map (section 5); never leaves this folder
customer-a-umwl-import.real.csv
customer-a-ipl-import.csv
runs/ # one subfolder per run
```

SITUATION	WHAT TO DO
e. Another account or another PCE	Repeat a–d in a new folder. Never copy <code>pce.yaml</code> or <code>anon-map.json</code> . The binaries can be shared: <code>ln -s ~/illumio/bin/umwl-tui umwl-tui</code> for the application, and for workloader either a symlink or the key <code>w</code> with the user scope (<code>~/.config/umwl-tui/config.json</code>), which every new folder picks up automatically.
f. Updating the tools	Download the new release (or <code>git -C kit pull && make -C kit build</code>). <code>umwl-tui --version</code> prints the version. For workloader, press <code>b</code> again: the existing clone is fetched and checked out at the newest tag and rebuilt.
Several PCEs in the same <code>pce.yaml</code>	Not the kit's scheme. If it exists anyway, always pass <code>--pce <name></code> to <code>umwl-tui</code> ; the preflight shows the <code>pce-list</code> summary and the working folder before going on.
Legacy fallback	<code>python3 kit/legacy/umwl_loader.py --setup-only</code> and <code>python3 kit/legacy/umwl_loader.py <csv> --ipl ... --priority 1</code> follow the same steps in a plain terminal.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — `cmd/root.go`: the configuration file is `--config-file`, else `ILLUMIO_CONFIG`, else `./pce.yaml` in the current directory — github.com/brian1917/workloader/blob/master/cmd/root.go
2. `workloader` — `cmd/pcemgmt/addpce.go`: `pce-add` with `--api-key` takes `--api-user`, `--api-secret` and `--org`; without it, it asks for email and password — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. GitHub CLI — `gh auth login`, `gh release download`, `gh repo clone` — docs.github.com/en/github-cli/github-cli/about-github-cli

■ Exporting from the PCE

Three kinds of data feed the analysis. The traffic export is mandatory; the workload, label and IP list exports make the proposals reuse the customer's existing nomenclature and avoid duplicates. The console pictures in this section are **schematics**: they show where the controls are and what to expect; exact labels vary slightly between console versions (the steps follow the official documentation for Illumio Core 23.5–25.x). In 22.x–23.x consoles the view is called **Explorer**; in 24.x/25.x the same data is in the **Traffic** view under **Explore**¹.

4.1 Traffic export (Explore › Traffic) — mandatory

PCE CONSOLE — EXPLORE › TRAFFIC: FILTERS, TIME RANGE, RESULTS SETTINGS, EXPORT

PCE console — Explore › Traffic schematic, not a screenshot; labels vary by version

Dashboard
Explore
Map
Traffic
Mesh
App Groups
Servers & Endpoints
Policy Objects
Labels
IP Lists
Services
Policies
Settings

2 Source: any → Destination: label Ordering · Service: any 4 Run

3 Time: Last 7 days ▼ (custom) 5 Reported ▼ | Draft More ▼ › Results Settings Load Results

Source	Destination	Port/Proto	Process	Flows	Policy decision	First / Last
10.20.1.11	ord-app-01...	8080 TCP	java	21,025	Unenforced Deny	Sep 1 → Sep 2
10.10.4.21	ord-app-01...	10050 TCP	zabbix_agentd	24,751	Unenforced Deny	Sep 1 → Sep 2
10.30.1.21	bil-app-01...	7001 TCP	java	170,965	No Rule	Aug 26 → Sep 2
bil-app-01	169.254.169.254	53 UDP	—	5,225,842	Allowed	Aug 26 → Sep 2
...	...	...	...	...	...	...

Showing 5,000 of 5,000 results — exactly the cap: raise it in More › Results Settings (step 5)

6 ↓ Export

- 1. Explore › Traffic** (Explorer in older consoles). Expected: the filter bar (Source / Destination / Service), the time selector and an empty results table.
- 2. Filters.** Put the workloads in scope as Destination and again as Source in a second query, or filter by their labels (for example the Application label of the POC group) and leave the other side open. Do not filter by port. Add exclusions (**More › Show Exclusion Filters**) for known vulnerability scanners once identified: a single scan can consume the whole results budget.
- 3. Time range.** Seven days is the right first pass (one full weekly cycle: backups, batch jobs, patching). Longer windows hit the results cap sooner; run a second, longer query for low-frequency flows if needed.
- 4. Run.** Wait for the query to complete. Expected: the table fills and the footer shows "N of M results". Queries from the last 24 hours reappear under **Load Results** without re-running¹.
- 5. More › Results Settings.** Raise the maximum results before exporting. The console shows up to 100,000 rows and the export can reach 200,000^{1, 5}; the default of a saved query can be as low as 5,000. If the export has exactly the cap number of rows, it is truncated: the analysis will say what consumed it and which filter to add.
- 6. Export.** The button downloads a CSV (`TrafficData <date>.csv`) with one column per label type²; the full header is in Appendix A.

Checks on the file before anything else. Open it as text (any editor, `head`, Python). Never open it in Excel and save: Excel strips the dots of IP addresses that look like numbers (`10.20.2.20` becomes `1020220`) and rewrites dates. If someone already did, keep the file anyway: the analysis reconstructs the IPs deterministically, but say so in the prompt. Count the rows against the cap of step 5. Note the date format (the export mixes `MM/DD/YYYY HH:MM` and, in some versions, `YYYY-DD-MM` ; the analysis handles both, but state the window you expect). The `Reported by` column tells which side had the VEN; the `Enforcement` columns tell which IPs are workloads. The policy decision column carries Allowed, Blocked and Potentially Blocked semantics³.

TERMINAL · ALTERNATIVES TO THE CONSOLE EXPORT

```
# workloader: "Export traffic data"; dates in GMT, --max-results default 100000, maximum 200000
./workloader traffic --start 2026-08-17 --end 2026-09-01 --max-results 200000 --output-file TrafficData.csv
# REST API (the same asynchronous query the console runs; max_results up to 200,000):
# POST /api/v2/orgs/<org>/traffic_flows/async_queries → GET .../async_queries/<uuid> → download
```

The columns of the `workloader traffic` CSV are not identical to those of the console export; the analysis instructions normalize column names, so either one works.

4.2 Workloads, labels and label types — recommended

PCE CONSOLE — SERVERS & ENDPOINTS › WORKLOADS, AND THE TERMINAL EQUIVALENT

PCE console — Servers & Endpoints › Workloads schematic, not a screenshot; labels vary by version

Dashboard
Explore
Servers & Endpoints **1**
Workloads
VENs
Policy Objects
Policies
Settings

Filter: Enforcement · Labels · Managed / Unmanaged ... Add ▾ **2** ↓ Export

Name	Hostname	Interfaces	Enforcement Role	App	Env	Loc
ord-app-01	ord-app-01.lab...	eth0:10.20.2.20	Visibility Only	App	Ordering	Production DCI
Front-End Ord. 10.20.1.11	—	eth0:10.20.1.11	(unmanaged)	Web	Ordering	Production DCI
DNS Corporativo 10.10.0.53	—	eth0:10.10.0.53	(unmanaged)	DNS	Shared Services	Production DCI

Terminal equivalent (recommended, no clicks):

```
./workloader wkld-export --output-file pce-workloads.csv
./workloader label-export --output-file pce-labels.csv
./workloader label-dimension-export --output-file pce-label-types.csv
```

- Servers & Endpoints › Workloads** lists managed (VEN) and unmanaged workloads with their labels; the filter narrows by enforcement, labels and managed/unmanaged.
- Export** downloads the list as CSV: the same data as `workloader wkld-export`, which is what `umwl-tui` uses in step 2. Expected file: one row per workload with `href`, `hostname`, `name`, `interfaces`, `managed` and the labels⁴; the proposals are reconciled against it by IP.

From the working folder, once `pce.yaml` exists (`./umwl-tui --setup-only` creates it), the three terminal commands of the picture produce `pce-workloads.csv` (one row per workload), `pce-labels.csv` (`href`, `key`, `value` : the existing values per type) and `pce-label-types.csv` (one row per label type). Console equivalents: **Servers & Endpoints › Workloads › Export**, **Policy Objects › Labels** (filter by type, Export) and **Settings › Label Settings** for the types. A CSV column in the proposals that is not a type here is silently ignored by workloader; create the type first (`workloader label-dimension-import`, section 11).

4.3 Labels, label types and IP lists — optional but useful

PCE CONSOLE — POLICY OBJECTS › LABELS, SETTINGS › LABEL SETTINGS, POLICY OBJECTS › IP
LISTS

PCE console — Policy Objects › Labels › IP Lists › Settings › Label Settings schematic, not a screenshot; labels vary by version

Dashboard
Explore
Servers & Endpoints
Policy Objects
Labels
Label Groups
IP Lists
Services
Settings
Label Settings

Labels — filter by type: role · app · env · loc ↓ Export

role	Web, App, Database, LoadBalancer, DNS ...
app	Ordering, Billing, Shared Services ...
env	Production, Staging
loc	DCI, Cloud

Terminal equivalent:
`./workloader label-export --output-file pce-labels.csv`
`./workloader label-dimension-export --output-file pce-label-types.csv`
`./workloader ipl-export --output-file pce-iplists.csv`

- Policy Objects › Labels:** the existing values per type. Export them so the analysis reuses the customer's own values and naming style.
- Settings › Label Settings:** the label types. A CSV column that is not a type is ignored by workloader; create the type first.
- Policy Objects › IP Lists:** the existing lists (corporate networks, DNS, NTP). `./workloader ipl-export --output-file pce-iplists.csv` or **Policy Objects › IP Lists › Export**. Attach it so the proposed rules reference existing lists instead of creating duplicates.

Screenshots of the nomenclature. Two or three screenshots of existing unmanaged workloads and labels in the console (name pattern, label prefixes, interface format) let the assistant copy the customer's conventions exactly. The kit imposes no label convention of its own (plain values as Illumio recommends, or whatever style the PCE already uses); its defaults are workload name `"<descriptive role> <IP>"`, empty hostname and interface `eth0:<ip>` (section 7).

SOURCES — OFFICIAL DOCUMENTATION

- Illumio Core 23.5 — Visualization — About the Visualization Tools (Explore › Traffic, filters, Results Settings, 100,000 UI / 200,000 export limits, Load Results) — product-docs-repo.illumio.com/Tech-Docs/Core/23.5/Visualization/out/en/visualization/visualization-tools/about-the-visualization-tools.html
- Illumio Core 24.2 — Visualization — Traffic Table (Export to CSV, one column per label type) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Visualization/out/en/visualization-tools/traffic-table.html
- Illumio Core 23.2 — Visualization — Traffic Table (Allowed / Blocked / Potentially Blocked semantics) — product-docs-repo.illumio.com/Tech-Docs/Core/23.2/Visualization/out/en/visualization-user-guide-23-2/visualization-tools/traffic-table.html
- workloader — `cmd/wkldexport/headers.go`: export columns (hostname, name, interfaces, public_ip, href, managed, ven_href...); README: traffic, label-export, label-dimension-export, ipl-export — github.com/brian1917/workloader/blob/master/cmd/wkldexport/headers.go
- Illumio Core 22.5 — REST API — Explorer: traffic_flows/async_queries, max_results 200,000 — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html

■ Anonymization

The exports leave the working folder to be analyzed. Before they do, everything that identifies the customer is replaced by consistent tokens, and only the evidence the analysis needs stays intact. Private IPs are the join key between the export, the proposals and the PCE inventory: they are never anonymized.

What leaves the folder and what must not

THE ASSISTANT NEEDS	THE ASSISTANT DOES NOT NEED
Private IPs (exact), ports, protocols, process names, service-account usernames (<code>root</code> , <code>oracle</code> , <code>zabbix</code> ... they are role evidence), flow counts, dates, label values, and the shape of the hostnames (which host is which, which tier a domain suggests).	The customer's name, real hostnames and FQDNs, personal usernames, the customer's public IPs, or anything from <code>pce.yaml</code> .

NEVER UPLOAD

`pce.yaml` (API key), `anon-map.json` (the pseudonym map), `runs/` (inventories with real names), API keys or session tokens pasted from a terminal.

The tool: `anonymize_export.py`

Consistent and reversible pseudonymization, Python standard library only. The same input always maps to the same token, across every column and every run that shares the map file, so the traffic export and the workloads export can be anonymized separately and still agree.

TERMINAL · BEFORE THE ANALYSIS (WORKING FOLDER; THE MAP IS CREATED OR UPDATED IN ANON-MAP.JSON)

```
python3 kit/anonymize_export.py anon "TrafficData 9_2_2026.csv" -o traffic.anon.csv --map anon-map.json \
--customer "Acme S.A." --customer acme --domain acme.com --domain acme.local --public-ips
python3 kit/anonymize_export.py anon pce-workloads.csv -o pce-workloads.anon.csv \
--map anon-map.json --domain acme.com

# after the analysis: restore the real names inside the proposed CSVs (descriptions, FQDNs) before loading
python3 kit/anonymize_export.py deanon G1-umwl-import.csv -o G1-umwl-import.real.csv --map anon-map.json
```

WHAT IT REPLACES	WITH
Hostnames and FQDNs in the Name, Hostname and FQDN columns	<code>host-0001.company.com</code> style, keeping the domain depth (<code>unix.corp.acme.com</code> becomes <code>unix.dept.company.com</code>) so tiers remain recognizable.
The customer's name (<code>--customer</code> , repeatable, case-insensitive whole words)	<code>Cliente</code> , everywhere.
Domains given with <code>--domain</code> and their sub-domains	<code>company.com</code> , <code>dept.company.com</code> . Without <code>--domain</code> , every domain seen is pseudonymized.
Personal usernames (Consuming Username, Username)	<code>user-01</code> ...; well-known service accounts are kept because they carry the role evidence.
The customer's public IPs (only with <code>--public-ips</code>)	<code>203.0.113.x</code> / <code>198.51.100.x</code> test ranges. RFC 1918, CGNAT and link-local addresses stay intact.
Untouched	Private IPs, ports, protocols, processes, label values, flow counts, dates.

ALWAYS PASS --DOMAIN WITH THE CUSTOMER'S DOMAINS

Third-party FQDNs (`*.trendmicro.com` , `*.oraclecloud.com` , `*.googleusercontent.com`) then stay visible, and they are evidence: EDR consoles, cloud metadata, SaaS. The map file is written with mode 600 and is in `.gitignore` .

Leak check. The tool ends by scanning the output for real domains, the customer's name or real short hostnames still present, and prints a warning if it finds any. Read the warning: if a hostname is also used as a label value or inside a description, add it with `--customer` so it is replaced everywhere.

What anonymization changes in the analysis

Roles must come from ports, processes and users, never from names: the assistant is instructed accordingly in `context.md` , and that is also why the workload names in the proposals are "role + IP". The `description` of every proposed workload carries the pseudonymized FQDN ("FQDN en export (puede estar anonimizado): `host-0012.dept.company.com`"); `deanon` restores it before loading, so the PCE object shows the real name in its description while the report shared with third parties does not. Label values are not anonymized: they are the customer's nomenclature and must match the PCE. If a label value itself contains the customer's name, rename it in the PCE first or add it to `--customer` .

SOURCES — REPOSITORY

1. `illumio-workloader-import-kit` — `anonymize_export.py` (`anon` / `deanon`, `--customer`, `--domain`, `--public-ips`, `leak check`) and `.gitignore` — github.com/roschereric/illumio-workloader-import-kit

■ Running the analysis with Claude

A Claude Project keeps instructions and reference files across conversations, which is exactly what this workflow needs: every new export is a new conversation, but the parsing rules, the CSV contracts and the nomenclature must not be re-explained each time. The method lives in two files of the repository: `docs/prompts/context.md` (the full method: inputs, normalization, the workload-versus-IP-list criterion including the load-balancer rule, the label model, the findings format, the deliverables and their CSV contracts, the v2 procedure and the quality gate) and `docs/prompts/prompt-short.md` (the per-conversation message, v1 and v2 variants).

Project setup (once)

1

Create the project

claude.ai › Projects › New project. Name it for the practice, not for a customer (for example `Illumio - flow analysis & object proposals`): one project, one conversation per customer and export. If two customers use different nomenclatures, state the convention in the short prompt (the default) or create one project per customer.

2

Project instructions

Paste the full text of `docs/prompts/context.md`. When conventions change (a new label type, a different name pattern, a new site), edit the instructions once instead of repeating it in conversations.

3

Project knowledge (files)

Add `context.md` again as a file (so it can be cited), the fictional sample pair `examples/sample-umwl-import-v2.csv` and `examples/sample-ipl-import-v2.csv` as format references, and the header line of a traffic export (`docs/export-columns.txt`). Do not add real customer exports as permanent knowledge: attach them per conversation.

4

What never goes into the project

`pce.yaml`, API keys, `anon-map.json`, or the customer's name in the project title or description.

Per conversation

Attach to one message: the anonymized traffic export, the anonymized workloads export, the labels and label-types exports, the optional IP lists export, the optional console screenshots, and the text of `prompt-short.md` with the group id, sites and scope lines edited. Name the conversation `<customer code> - <export date> - v1|v2`. Without a Project (for example from another person's account), attach `context.md` with every request and start the message with "Follow context.md"; the results are the same, only the convenience differs. Ask for corrections in the same conversation ("10.30.1.x are front-end servers, not the LB; the LB is the VIP"); the assistant regenerates the CSV and the report. Then run `deanon` on the CSVs (section 5).

What comes back, and what to check before accepting it

OUTPUT	CHECK
Report (single HTML file, Spanish, no customer name)	Executive numbers equal the tables; the truncation verdict; the tier diagram matches what you know of the architecture.
<code><group>-umwl-import.csv</code>	Role <code>LoadBalancer</code> only on VIPs; front-ends behind a VIP are <code>Web</code> ; label values reuse the PCE's (same spelling, same style, no invented prefixes); nothing inferred from hostnames; one row per IP; names unique; interfaces <code>eth0:<ip></code> ; <code>review</code> = PENDING.
<code><group>-ipl-import.csv</code>	CIDRs match the addressing plan; no duplicate of an existing IP list.
Findings S-xx	Each has evidence with counts and dates; scans versus direction artefacts distinguished; end-of-support claims cite the vendor.
Summary in the chat	Window, truncation, hosts with a VEN, how many workloads and lists it proposes, the three most important findings and what it needs you to confirm.

THE DELIVERABLES ARE IN SPANISH

The report and the CSV descriptions the prompts ask for are customer-facing and in Spanish, because that is how this kit is used with LATAM customers. To change the language, edit the "Inputs" and "Deliverables" sections of `context.md`.

SOURCES — REPOSITORY

1. illumio-workloader-import-kit — [docs/prompts/context.md](#) (project instructions), [docs/prompts/prompt-short.md](#) (per-conversation message), [docs/export-columns.txt](#), [examples/*-v2.csv](#) — github.com/roschereric/illumio-workloader-import-kit

■ Policy objects and naming convention

`umwl-tui` creates two kinds of object and, as a side effect, label values. Knowing what each one is explains the decisions of the analysis and those of the application.

OBJECT	WHAT IT IS AND WHEN IT IS USED
Unmanaged workload (UMWL)	A network entity without a VEN that is registered in the PCE so that rules can be written about it; policy is enforced by the rules on the side that has a VEN ¹ . For specific servers with an identifiable role (DNS, NTP, Zabbix, Splunk, NetBackup, databases, load balancers, bastions, scanners). One per IP, with labels.
IP list	A static collection of addresses, ranges and FQDNs ² . For broad peers or peers that are not servers : user VLANs, whole subnets, cloud metadata (169.254.169.254), public NTP, SaaS consoles. Also as a shortcut to write a rule today that migrates to labels later.
Labels and label types	Four default types (Role, Application, Environment, Location) plus any created in the PCE, up to 20, each with a unique key ³ . Values are free text; Illumio's guidance is to keep them simple and to mirror the names the organization already uses (<code>Web</code> , <code>Database</code> , <code>Production</code> , <code>AWS</code>) — there is no mandated prefix or separator scheme ⁴ . workloader creates new values, not types : a column that is not an existing type is silently ignored (step 3 of <code>umwl-tui</code> detects it).
Services	Port and protocol objects. The report proposes them for the rules; the kit does not create them (they are loaded with <code>svc-import</code> or from the console).

WHY THE PRIORITY GOES IN THE DESCRIPTION

It is the only free-text field that travels with the object to the PCE and can be read afterwards in the console. `umwl-tui` uses it to filter (`--priority 1`) without needing extra columns that workloader would ignore.

Naming convention

FIELD	RULE	EXAMPLE
Labels	One CSV column per label type <i>key</i> (<code>role</code> , <code>app</code> , <code>env</code> , <code>loc</code> , custom keys); the value is the label value exactly as it exists in the PCE (<code>label-export</code>), or a new one in the same style. The kit imposes no convention of its own: if the customer prefixes or capitalizes values in a certain way, follow it; if the PCE is empty, use plain values as Illumio recommends. Role = function of the host (<code>Web</code> , <code>App</code> , <code>Database</code> , <code>LoadBalancer</code> only on VIPs — the front-ends behind a VIP are <code>Web</code> —, <code>DNS</code> , <code>Monitoring</code> , <code>Backup</code> ...); Application = the business application that owns the host, with its own values for shared infrastructure (<code>Shared Services</code> , <code>Monitoring</code> , <code>Security Tools</code> , <code>Admin Access</code>); Environment <code>Production</code> / <code>Staging</code> ; Location = site or region as the customer names it	<code>DNS · Shared Services · Production · DC1</code>
<code>name</code>	Descriptive role followed by the IP; it is what the console shows and what makes the row unique	<code>Zabbix Server 10.10.4.21</code>
<code>hostname</code>	Empty unless the real hostname is known. The FQDNs in the exports may be pseudonymized; an invented hostname is misleading	(blank)
<code>interfaces</code>	<code>eth0:<ip></code> ; several interfaces separated by <code>;</code>	<code>eth0:10.10.4.21</code>
<code>description</code>	Prefix <code>[<group> P<priority> conf:<Alta Media Baja>]</code> followed by the evidence comment and the FQDN seen in the export	<code>[G1 P1 conf:Alta] Zabbix server: polls 10050/TCP...</code>
IP lists, services, rulesets	Descriptive names in the customer's style; the IP list description starts with <code>[<group>]</code> and ends with the intended rule usage	<code>Corporate DNS · Ordering Web 8080 · Ringfence Ordering</code>

review

Working column: PENDING in v1; PENDING /
UPDATED / UNCHANGED in v2. workloader
ignores it

PENDING

SOURCES — OFFICIAL DOCUMENTATION

1. Illumio Core 24.5 — Security Policy — Workload Setup Using PCE Web Console (Add Unmanaged Workload) — product-docs-repo.illumio.com/Tech-Docs/Core/24.5/Security-Policy/out/en/security-policy-guide-24-5/workloads/workload-setup-using-pce-web-console.html
2. Illumio Core 24.2 — Getting Started — Policy Objects — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting-Started/out/en/policy-overview/policy-objects.html
3. Illumio Core 25.4 — Security Policy — Create a Label Type — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Security-Policy/out/en/security-policy-objects/about-labels-and-label-groups/label-types/create-a-label-type.html
4. Illumio Core 24.2 — Getting Started — Labeling Workloads lesson (example values Web, Database, production, staging, AWS; mimic infrastructure names) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Getting+Started/out/en/application-ringfencing/labeling-workloads-lesson.html

■ umwl-tui step by step

Two commands cover normal use, always from the working folder (section 3). The first runs once per folder; the second, once per CSV, after `deanon`.

TERMINAL · WORKING FOLDER OF THE ACCOUNT + PCE

```
# 1. once per account + PCE: workloader, pce.yaml (pce-add --api-key) and the connection test
./umwl-tui --setup-only

# 2. load the priority-1 workloads and, at the end, the IP lists
./umwl-tui customer-a-umwl-import.real.csv --ipl customer-a-ipl-import.csv --priority 1

# other flags: --pce NAME · --workloader PATH (default ./workloader, then the PATH)
#               --chunk 20 (rows per wkld-import call) · --runs ./runs · --priority 1,2 · --version
```

The screen

The layout is fixed: a **status bar** on top with the state, the PCE name, the run folder and the step counter; a **sidebar** on the left with the ten steps and their state (✓ done, ► active, ○ pending, ✕ failed, – skipped) and the recent events; the **panel** of the active step on the right (tables browsed with the arrow keys, side-by-side "proposed versus in the PCE" detail, progress bars); the **log pane** at the bottom with the live workloader output, each command echoed with its arguments; and a **key bar** that changes per step. Every decision that matters is a modal: choice modals (one key per option, `esc` cancels), forms (`tab` between fields, `enter` submits) and information boxes. Destructive confirmations have a red title.

UMWL-TUI · STEP 4 (RECONCILE BY IP)

umwl-tui ● Ready

PCE default (pce.yaml) | run 20260903-042451 | step 4/10

STEPS

- ✓ 0 Preflight
- ✓ 1 Load CSV
- ✓ 2 PCE inventory
- ✓ 3 Labels
- 4 Reconcile by IP
- 5 Review new
- 6 Dry run
- 7 Execute
- 8 Verify
- 9 IP lists
- 10 Report

EVENTS

04:24 reconcile: 21 NEW...

■ Reconcile by IP – 24 rows · 3 awaiting a decision

IP	PROPOSED NAME	STATE	IN THE PCE
► 10.10.4.21	Zabbix Server 10.10.4...	EXISTS-UNMANAGED	zbx-old
10.30.4.21	Zabbix Proxy Cloud 10...	NEW	
10.10.0.53	DNS Corporativo 10.10...	CONFLICT-MULTIPLE	dns1

...

■ Selected: 10.10.4.21

proposed	in the PCE	unmanaged
name Zabbix Server 10.10.4.21	name Zabbix Server	
role Monitoring	role –	
app Monitoring	app –	

workloader output · tab to scroll

04:24:51 \$ workloader wkld-export --output-file runs/.../pce-workloads.csv

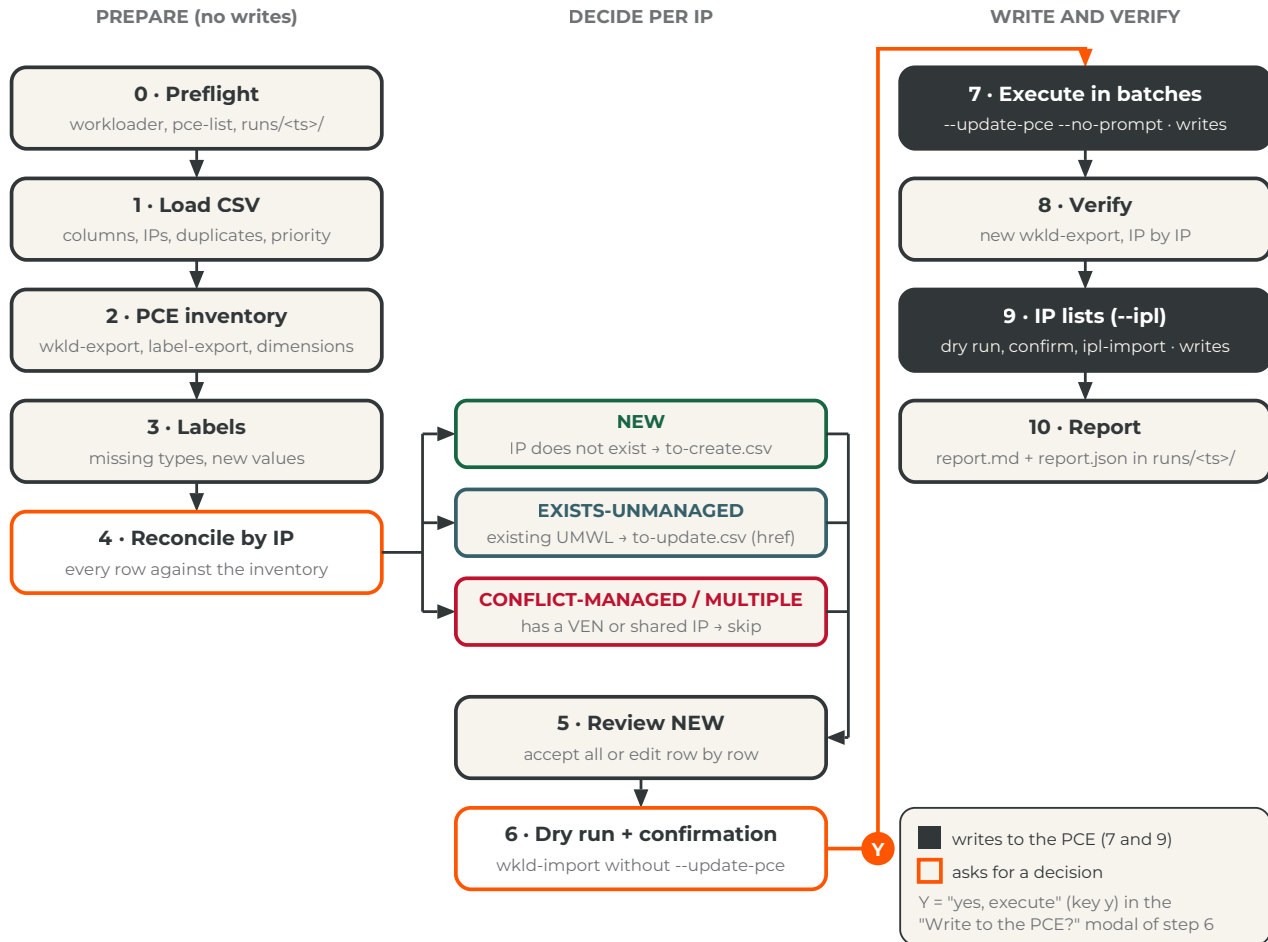
u update s skip c create anyway r rename+update U/S all of this kind enter next tab log ? help q quit

Global keys. `tab` moves the focus to the log pane (scroll with the arrows); `?` opens the help; `q` quits, with a confirmation once work has started; `ctrl+c` quits immediately. Tables accept `↑ ↓ j k pgup pgdown g G`. Modals capture every key while open.

The file chooser. Without a CSV argument, `umwl-tui` opens a Midnight Commander-style chooser after the preflight: an editable path field on top (`tab` or `/` to edit it, `~` expands, `enter` on a directory jumps there), the listing below with `..` first, then directories, then `.csv` files highlighted and other

files dimmed; `←` or backspace goes to the parent, `enter` or `→` opens or selects, `esc` cancels. It then asks for the optional IP lists CSV (`esc` = none) and for the priority filter. A CSV that fails validation shows the reason in a dialog, returns to the preflight and reopens the chooser.

SEQUENCE OF STEPS AND DECISION POINTS



What each step does and decides

STEP	WHAT HAPPENS	KEYS
0 Preflight	Locates workloader (flag → saved path → <code>./workloader</code> → PATH) and prints its version and CPU architecture, offers to build it from source or download it, runs <code>pce-list</code> and shows the PCE and the working folder, tests the connection with a read-only <code>label-dimension-export</code> , creates <code>runs/<ts>/</code> . All three checks must pass (an Intel binary under Rosetta is a warning, not a failure).	<code>enter</code> next · <code>b</code> build · <code>d</code> download · <code>w</code> path · <code>p</code> pce-add · <code>t</code> test · <code>r</code> re-check

1 Load CSV	Required columns (<code>hostname</code> , <code>name</code> , <code>interfaces</code>), valid IPs, repeated IPs, the <code>--priority</code> filter, duplicate hostnames or names (the IP is appended as a suffix), label columns found. Warnings and dropped rows are listed above the table.	<code>enter</code> inventory · <code>↑↓</code> browse · <code>o</code> other CSV
2 PCE inventory	<code>wkld-export</code> , <code>label-export</code> , <code>label-dimension-export</code> into <code>runs/<ts>/</code> ; every PCE workload is indexed by its IPs (<code>interfaces</code> and <code>public_ip</code>) and the managed ones are marked (<code>managed</code> or <code>ven_href</code>). Read-only; goes on automatically.	—
3 Labels	CSV columns that are not a label type of this PCE are flagged: drop them or quit and create the type with <code>label-dimension-import</code> . The values that workloader will create are listed and confirmed.	<code>enter</code> accept · <code>d</code> drop unknown columns
4 Reconcile by IP	Every row is classified: NEW (the IP does not exist), EXISTS-UNMANAGED (the IP belongs to a UMWL), CONFLICT-MANAGED (the IP belongs to a workload with a VEN), CONFLICT-MULTIPLE (several workloads share the IP). The detail shows the proposed row next to the PCE object. Updating a managed workload asks for an extra confirmation because it can change its policy immediately; <code>U</code> never touches CONFLICT-MANAGED rows. Rows left undecided at <code>enter</code> are skipped after a modal.	<code>u</code> update · <code>s</code> skip · <code>c</code> create anyway · <code>r</code> rename + update · <code>U</code> / <code>S</code> all of this kind · <code>n</code> next undecided · <code>enter</code> next
5 Review new	Table of the workloads to create, with the update and skip counts; edit name, hostname, description and labels row by row, or skip a row. Writes <code>to-create.csv</code> , <code>to-update.csv</code> and <code>skipped.csv</code> .	<code>enter</code> accept all → dry run · <code>e</code> edit · <code>s</code> skip
6 Dry run	<code>wkld-import to-create.csv --umwl --update=false --match name</code> and <code>wkld-import to-update.csv --match href</code> , both without <code>--update-pce</code> ; the relevant log lines are shown. <code>enter</code> opens the red "Write to the PCE?" modal with the counts; only <code>y</code> writes.	<code>enter</code> execute · <code>x</code> stop + report · <code>↑↓</code> scroll
7 Execute	Batches of <code>--chunk</code> rows, one <code>wkld-import ... --update-pce --no-prompt</code> call each, create batches first and update batches next, with a progress bar and the first and last IP of each batch. A failed batch opens a modal: retry, skip (recorded) or abort.	modal: <code>r</code> retry · <code>s</code> skip · <code>q</code> abort
8 Verify	A new <code>wkld-export</code> ; every created IP must appear in the inventory. Shows created, updated, labels created and failed batches.	<code>enter</code> next
9 IP lists	Only with <code>--ipl</code> : <code>ipl-import</code> dry run, then a confirmation modal, then <code>ipl-import ... --update-pce --no-prompt</code> .	<code>enter</code> import · <code>n</code> skip
10 Report	<code>runs/<ts>/report.md</code> and <code>report.json</code> : status, created, updated, skipped, labels created, failed batches, verification and every workloader command with its exit code.	<code>enter</code> or <code>q</code> finish

READING THE DRY RUN: "THE MATCH COLUMN CANNOT BE BLANK" AND "NOTHING TO BE DONE"

workloader matches each row on one column, chosen by `--match` or, without it, by priority over the columns present in the CSV: `href`, then `hostname`, then `name`. A row whose match column is blank is skipped with "the match column cannot be blank"; there is no fallback from hostname to name, and a CSV where every row is skipped ends with "nothing to be done"³. With the blank-hostname convention of section 7, a plain `wkld-import` would pick `hostname` and skip every row. That is why `umwl-tui` passes `--match name` on the create pass whenever a row has a blank hostname and `--match href` on the update pass, and why a dry run containing either message, a "[WARN]" or an "[ERROR]" is reported as not ok: the "Write to the PCE?" modal then says so. A batch that does nothing is a wasted run; a skipped row is a row the PCE will not get.

EVERYTHING THE RUN PRODUCED

`runs/<timestamp>/` holds the PCE inventories before and after, `to-create.csv`, `to-update.csv`, `skipped.csv`, one CSV and one workloader log per batch (`create-chunk001.csv`, `create-chunk001.log` ...), the dry-run logs, `report.md` and `report.json`. Run folders are 0755 and CSVs 0644: they contain customer IPs but no secrets. Quitting at the dry run with `x` still writes the report, marked "stopped before writing to the PCE".

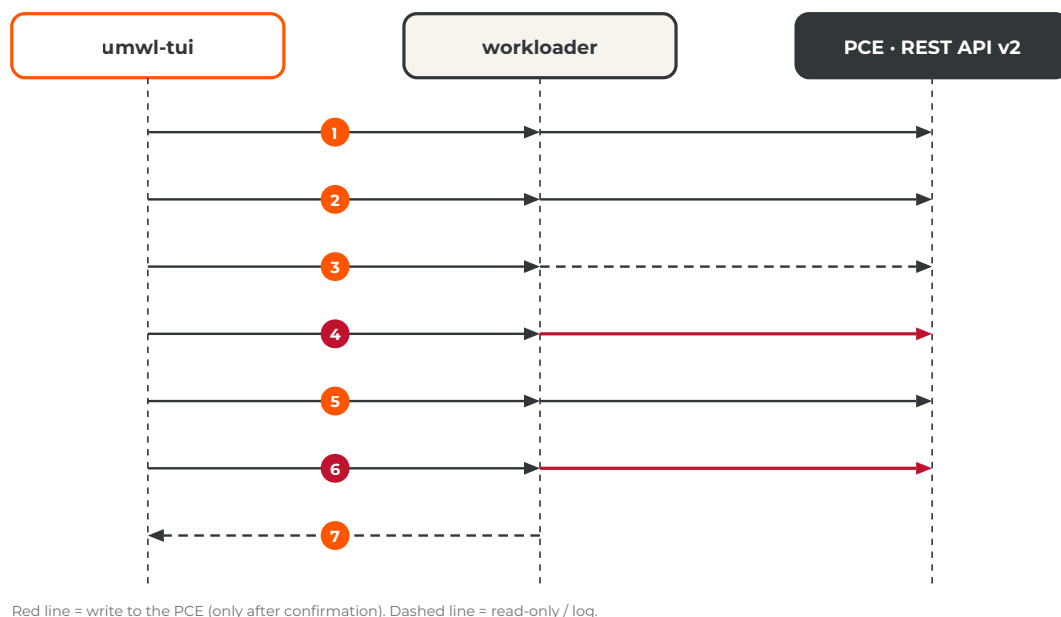
SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — README: `--update-pce` and `--no-prompt` ("auto-accept this prompt, as would be needed in automation"); logs in `workloader.log` — github.com/brian1917/workloader
2. `workloader` — `cmd/root.go`: persistent flags `--pce`, `--log-file`, `--update-pce`, `--no-prompt`, `--config-file` — github.com/brian1917/workloader/blob/master/cmd/root.go
3. `workloader` — `cmd/wkldimport/cmd.go`: `--match (href, hostname, name, external_data)`, match column priority, "the match column cannot be blank", `--umwl`, `--update` — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go

■ umwl-tui and workloader

`umwl-tui` never talks to the PCE API directly: it runs workloader subcommands (the `./workloader` binary in the working folder, or the one on the PATH, or the one given with `--workloader`) with `exec` and typed arguments, no shell in between, and reads their logs. Every call runs with `--log-file runs/<ts>/<step>.log` and, if specified, with `--pce <name>`. Without `--update-pce`, no workloader command modifies the PCE¹; the application adds that flag (together with `--no-prompt`) only in steps 7 and 9, after the explicit confirmation modal.

WHO CALLS WHOM: UMWL-TUI, WORKLOADER AND PCE



1. `pce-list` (and `pce-add --api-key ...` from the preflight form when no PCE is configured); the connection test is a `label-dimension-export`.
2. `wkld-export`, `label-export`, `label-dimension-export` into `runs/<ts>/`: inventory (with `managed`, `ven_href`, `interfaces`, `public_ip`), labels and types. Read-only.
3. Dry run: `wkld-import to-create.csv --umwl --update=false --match name` and `wkld-import to-update.csv --match href`, without `--update-pce`; workloader logs the plan and modifies nothing.
4. Per batch: the same two commands on `create-chunkNNN.csv` / `update-chunkNNN.csv` plus `--update-pce --no-prompt`. Creates UMWLs, updates labels and description of existing ones, creates missing label values.
5. `wkld-export` again: every created IP must appear in the inventory.
6. With `--ipl: ipl-import file.csv` (dry run) and, after confirmation, `ipl-import file.csv --update-pce --no-prompt`.
7. Every call leaves its log in `runs/<ts>/`; the "to be created", "to be changed", "created new ... label", "bulk create/update workload successful", "[WARN]" and "[ERROR]" lines are shown on screen and kept in the report.

Update semantics

A row decided as *update* is rewritten before the dry run with the `href` of the PCE object, the `hostname` from the PCE, blank `interfaces` (workloader leaves interfaces untouched when the cell is blank) and the PCE `name` unless it was renamed. That is what makes a plain `wkld-import --match href` (no `--umwl`) update labels and description only.

The same steps without umwl-tui

TERMINAL · MANUAL EQUIVALENTS (SAME WORKING FOLDER, SAME PCE.YAML)

```
# PCE (once per account + PCE) and inventory
./workloader pce-add --api-key --name customer-a --fqdn pce.customer-a.example --port 443 \
  --api-user api_xxx --api-secret '...' --org 1
./workloader wkld-export --output-file pce-workloads.csv
python3 kit/legacy/reconcile_umwl.py pce-workloads.csv customer-a-umwl-import.real.csv

# dry run (read workloader.log), then write - blank hostnames need --match name
./workloader wkld-import customer-a-umwl-import.real-to-create.csv --umwl --match name
./workloader wkld-import customer-a-umwl-import.real-to-create.csv --umwl --match name --update-pce

# existing ones: labels and description only, matched by href
./workloader wkld-import customer-a-umwl-import.real-existing.csv --match href --update-pce
./workloader ipl-import customer-a-ipl-import.csv --update-pce
```

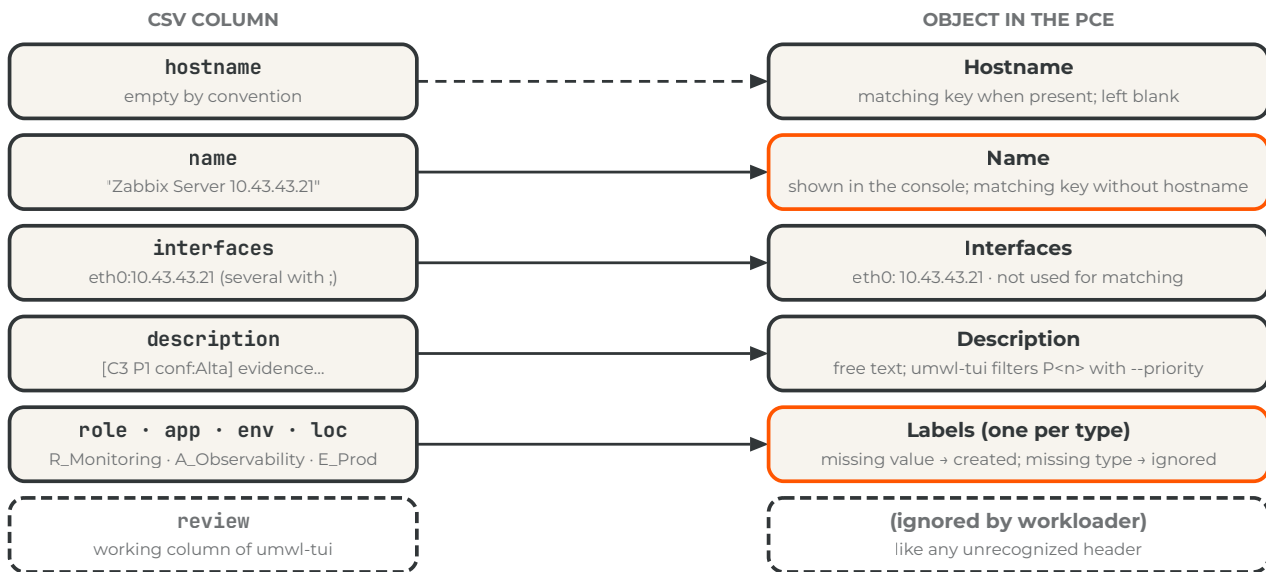
PCE-ADD NEEDS --API-KEY TO USE AN API KEY

In the workloader code, `--api-user`, `--api-secret` and `--org` are only read when the `--api-key` flag is present; without it, the command asks for email and password². The preflight form of `umwl-tui` always adds it.

From the CSV to PCE objects

`wkld-import` recognizes a fixed set of headers (`href`, `hostname`, `name`, `interfaces`, `public_ip`, `description`, `enforcement`, `visibility` and others) plus **one column per existing label type**; everything else is ignored³. The diagram shows which workload field each column goes to.

CSV COLUMNS AND UNMANAGED WORKLOAD FIELDS



Matching: by href, hostname or name, never by IP. Before creating, workloader checks whether the row already exists on one column: `href` if the column is present, else `hostname`, else `name`, or the one forced with `--match`³. The IP plays no part. That is why two rows with the same `name` and no `hostname` are the same workload for workloader (step 1 appends the IP as a suffix), and why a UMWL that already exists under another name would be created as a duplicate unless it is first reconciled by IP against the `wkld-export`: that is step 4 of `umwl-tui` and the only job of `legacy/reconcile_umwl.py`.

IP lists. The `ipl-import` contract is `name,description,include,exclude,fqdns`: `name` is the matching key when there is no `href` (if it exists, it updates; if not, it creates)⁴ and by convention it is lowercase with the `ipl-` prefix; `include` and `exclude` carry IPs, CIDRs or ranges separated by `;`; `fqdns`, names where applicable; `description`, the reasoning from the report.

SOURCES — OFFICIAL DOCUMENTATION

1. workloader — README: "When a command modifies resources in the PCE, workloader does not trigger the action unless the `--update-pce` flag is included" — github.com/brian1917/workloader
2. workloader — `cmd/pcemgmt/addpce.go`: pce-add flags (`--api-key`, `--api-user`, `--api-secret`, `--org`, `--disable-tls-verification`) — github.com/brian1917/workloader/blob/master/cmd/pcemgmt/addpce.go
3. workloader — `cmd/wkldimport/cmd.go`: recognized headers, interfaces format, `--umwl`, `--update`, `--match` (`href`, `hostname`, `name`, `external_data`) — github.com/brian1917/workloader/blob/master/cmd/wkldimport/cmd.go
4. workloader — `cmd/iplimport/iplimport.go`: headers `name`, `description`, `include`, `exclude`, `fqdns`; `;` separator; matching by `href` and `name` — github.com/brian1917/workloader/blob/master/cmd/iplimport/iplimport.go

■ Iterating: v2 exports

After the first rules are provisioned, export again with the same scope and a new window (section 4), anonymize it with the same `anon-map.json` (section 5) and run the v2 variant of `prompt-short.md` in a new conversation of the same Project, with the previous report and the CSVs already loaded or proposed attached. The v2 report compares window, rows and noise with the previous export, shows what became Allowed and what is still No Rule, lists the peers that were not seen in the new window (marked "sin tráfico en la ventana nueva", never deleted), and delivers the full CSVs again, not only the delta, with `review` = PENDING for new rows, UPDATED for rows whose comment or labels changed and UNCHANGED for the rest, plus a table of changes (IP, before, after, reason) and an updated findings section.

After `deanon`, `umwl-tui` does the rest: in step 4 the rows whose IP already exists come out as EXISTS-UNMANAGED and are updated by `href` (labels and description); the new ones are created. Re-loading the full CSV therefore only changes what changed. Name the conversation and the CSVs with the version (`<group>-umwl-import-v2.csv`) and keep the `runs/` folders of both loads in the customer folder.

SOURCES — REPOSITORY

1. [illumio-workloader-import-kit — docs/prompts/context.md](#) (v2 procedure) and [docs/prompts/prompt-short.md](#) (v2 variant); [examples/sample-umwl-import-v2.csv](#) — [github.com/roschereric/illumio-workloader-import-kit](#)

■ Troubleshooting and checklist

SYMPTOM	CAUSE AND FIX
macOS refuses to open <code>umwl-tui</code> or <code>workloader</code> ("the developer cannot be verified")	Gatekeeper quarantine on a downloaded file. <code>xattr -d com.apple.quarantine ./umwl-tui</code> (same for <code>./workloader</code>) and <code>chmod +X</code> . The preflight does this for workloader when it downloads it.
The check row says <code>amd64 binary on arm64 (runs under emulation/Rosetta 2)</code>	You have the upstream macOS release, which is Intel-only. Press <code>b</code> in the preflight to clone and build a native binary (needs <code>brew install go</code>), or follow block c' of section 3. Rosetta keeps working meanwhile (<code>softwareupdate --install-rosetta</code> if it is missing). <code>umwl-tui</code> itself has native arm64 and amd64 builds.
<code>b</code> says "Go toolchain missing"	<code>brew install go</code> (the dialog offers to run it when Homebrew is present) or the installer from go.dev/dl ; then <code>r</code> and <code>b</code> again. Without Go, <code>d</code> downloads the release.
<code>go build</code> fails with <code>proxy.golang.org ... 403</code> or "Host not in allowlist"	The network filters the Go module proxy. From <code>workloader-src/</code> run <code>GOPROXY=direct go build ...</code> (modules come straight from GitHub), or build on a machine with open egress and copy the file: it is a single static binary.
<code>umwl-tui</code> uses a different <code>workloader</code> than expected	Precedence: <code>--workloader</code> → <code>./umwl-tui.json</code> → <code>~/.config/umwl-tui/config.json</code> → <code>./workloader</code> → PATH. The check row prints "[path from ...]" when a settings file supplied it; <code>w</code> changes it, deleting the file resets it.

Dry run ends with "the match column cannot be blank" on every row and "nothing to be done"	workloader matched on <code>hostname</code> and every hostname is blank (section 8). <code>umwl-tui</code> passes <code>--match name</code> on the create pass and <code>--match href</code> on the update pass and reports that dry run as not ok. By hand: <code>wkld-import to-create.csv --umwl --match name</code> .
The file chooser does not show my CSV	It lists the current folder: type the path in the field on top (<code>tab</code> or <code>/</code>), use <code>..</code> to go up, or pass the CSV on the command line. Files with another extension are dimmed but selectable.
"Load CSV" fails and the chooser reopens	Required columns missing (<code>hostname</code> , <code>name</code> , <code>interfaces</code>), an invalid IP in <code>interfaces</code> , or no row matches <code>--priority</code> . The reason is shown in a dialog and in the log pane; fix the CSV and pick it again from the chooser (or press <code>o</code> in step 1 to choose another file).
<code>pce-add</code> asks for email and password	<code>workloader pce-add</code> was run by hand without <code>--api-key</code> ; without that flag workloader ignores <code>--api-user</code> , <code>--api-secret</code> and <code>--org</code> and attempts a username/password login. The preflight form always adds <code>--api-key</code> .
The dry run works and the import returns 401/403	The API key inherits the user's role. A global role with write access is needed (Global Organization Owner or Global Administrator; a scoped Workload Manager can create workloads but not new labels or IP lists) or an equivalent service account. Check the <code>org id</code> too.
<code>pce-list</code> shows another customer or another organization	Wrong folder (the current directory decides which <code>pce.yaml</code> is used) or <code>ILLUMIO_CONFIG</code> pointing to another file. One folder per account + PCE; <code>unset ILLUMIO_CONFIG</code> ; <code>--pce <name></code> only if the <code>pce.yaml</code> has several profiles.
The dry run says it will update a workload you did not expect	Match by <code>name</code> with an existing object. Check <code>runs/<ts>/dry-create.log</code> ; rename the row in step 5, or let the IP reconciliation of step 4 resolve it.
Only one of several workloads with the same name was loaded	Same <code>hostname</code> (or same <code>name</code> without hostname) = same workload for workloader. Make the <code>name</code> unique; step 1 appends the IP as a suffix.
A label column was not applied (label type missing)	The type does not exist in the PCE and <code>wkld-import</code> ignores the column; step 3 flags it. Create it in the console (Settings > Label Settings) or with <code>./workloader label-dimension-import types.csv --update-pce</code> (CSV with <code>key</code> and <code>display_name</code>) ¹ and run again.
<code>wkld-import</code> creates a label with different capitalization	Values are case-sensitive unless <code>--ignore-case</code> . Use exactly the values shown by <code>label-export</code> (attach it to the analysis, section 4.2).
Failed batch in step 7	Check <code>runs/<ts>/create-chunkNNN.log</code> ; the CSV of that batch stays in <code>create-chunkNNN.csv</code> to retry it from the modal or by hand with <code>wkld-import ... --umwl --match name --update-pce</code> .
The traffic export has exactly N round rows	It is truncated at the query limit. Raise the limit in Results Settings, filter by application, exclude scanners, shorten the window and export again; the console shows up to 100,000 results and the CSV up to 200,000 ^{2,3} .
The anonymizer warns about a leak	A real domain, the customer's name or a short hostname survived (often inside a label value or a description). Add it with <code>--customer</code> or <code>--domain</code> and run <code>anon</code> again with the same map.
TLS error with an on-prem PCE	Internal certificate. Install the CA in the keychain; the "Disable TLS verify" field of the preflight form (<code>--disable-tls-verification true</code>) only in a lab.

Before writing to the PCE

- ☐ **Right folder** — the current directory is the account + PCE folder, the preflight shows that PCE (FQDN and org) and `ILLUMIO_CONFIG` is not set.
- ☐ **Names restored** — `deanon` was run on the proposed CSVs and the descriptions show real FQDNs.
- ☐ **CSV in the v2 format** — empty hostname, unique `name` with IP, `interfaces` as `eth0:<ip>`, prefixed labels, priority in the description.
- ☐ **Label types** — every label column in the CSV exists in the PCE (step 3 without warnings).
- ☐ **Reconciliation reviewed** — no CONFLICT-MANAGED is updated without having looked at it; the EXISTS-UNMANAGED keep the PCE name unless explicitly renamed.
- ☐ **Dry run read** — the "to be created" lines match what is expected; no "cannot be blank", "nothing to be done", "[WARN]" or "[ERROR]".
- ☐ **Small batch the first time** — `--chunk 5` on the first run against a new PCE to limit the impact of an error.
- ☐ **Report archived** — `runs/<ts>/report.md` stays in the customer folder; it is not pushed to the repository or attached to a conversation.

SOURCES — OFFICIAL DOCUMENTATION

1. `workloader` — `cmd/labeldimension/import.go`: `label-dimension-import` (headers `key`, `display_name`, `href...`) — github.com/brian1917/workloader/blob/master/cmd/labeldimension/import.go
2. Illumio Core 25.4 — Visualization — About the Visualization Tools (Explore, Traffic, result limits, asynchronous queries) — product-docs-repo.illumio.com/Tech-Docs/Core/25.4/Visualization/out/en/visualization-tools/about-the-visualization-tools.html
3. Illumio Core 22.5 — REST API — Explorer: `traffic_flows/async_queries`, `max_results 200,000` — product-docs-repo.illumio.com/Tech-Docs/Core/22.5/REST-APIs/out/en/core-22-5-rest-api-developer-guide/visualization/explorer.html

■ Appendix A — Export columns

Header of the Traffic export of an Illumio Core 24.x/25.x console (the same line is in `docs/export-columns.txt`, meant for the project knowledge). Custom label types appear as additional `Source <Type> / Destination <Type>` columns¹.

DOCS/EXPORT-COLUMNS.TXT · TRAFFIC EXPORT HEADER

```
Source IP, Source IPIList, Source Name, Source Hostname, Source Enforcement,
Source Application, Source Environment, Source Location, Source Quarantine, Source Role,
Source Service Category, Source Service Role, Source FQDN, Source CSP ID, Source Account ID,
Source Tenant ID, Source Detail Type, Source Object Type, Source Category, Source Sub Category,
Source Region, Consuming Process, Consuming Service, Consuming Username,
Destination IP, Destination IPIList, Destination Name, Destination Hostname, Destination Enforcement,
Destination Application, Destination Environment, Destination Location, Destination Quarantine,
Destination Role, Destination Service Category, Destination Service Role, Destination FQDN,
Transmission, Destination CSP ID, Destination Account ID, Destination Tenant ID,
Destination Detail Type, Destination Object Type, Destination Category, Destination Sub Category,
Destination Region, Port, Protocol, Process, Service, Username, Num Flows, Bytes In, Bytes Out,
Connection State, Reported Policy Decision, Reported by, First Detected, Last Detected
```

COLUMN GROUP	WHAT THE ANALYSIS READS FROM IT
Source / Destination IP, Name, Hostname, FQDN	The peer identity. Private IPs are the join key; names and FQDNs are pseudonymized before the analysis.
Source / Destination Enforcement and label columns	Which side has a VEN (a value in Enforcement) and its labels; one column per label type.
Consuming Process / Service / Username, Process, Service, Username	Role evidence on the VEN side: which process opened or served the connection and as which user.
Port, Protocol, Transmission	The service and whether the traffic is unicast, broadcast or multicast.
Num Flows, Bytes In, Bytes Out, First / Last Detected	Volume and time pattern (continuous, scheduled, one-off).
Reported Policy Decision, Connection State, Reported by	Allowed, Blocked or Potentially Blocked as reported, and which side reported the flow.

SOURCES — OFFICIAL DOCUMENTATION

1. [Illumio Core 24.2 — Visualization — Traffic Table \(Export to CSV, one column per label type\) — product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Visualization/out/en/visualization-tools/traffic-table.html](https://product-docs-repo.illumio.com/Tech-Docs/Core/24.2/Visualization/out/en/visualization-tools/traffic-table.html)

■ Appendix B — Files in the repository

FILE	PURPOSE
<code>umwl-tui</code> (<code>cmd/umwl-tui</code> , <code>internal/</code> , <code>Makefile</code>)	The full-screen application (Go, Bubble Tea): source, build (<code>make build</code>), tests (<code>make test</code>), cross-compilation with checksums (<code>make dist</code>). Prebuilt binaries are attached to the GitHub releases.
<code>anonymize_export.py</code>	Pseudonymize exports (<code>anon</code>) and restore names in the proposals (<code>deanon</code>). Python standard library.
<code>docs/prompts/context.md</code>	Project instructions: the full method and the deliverable contracts.
<code>docs/prompts/prompt-short.md</code>	The per-conversation message (v1 and v2 variants).
<code>docs/export-columns.txt</code> , <code>docs/img/*.svg</code>	Header line of a traffic export (Appendix A); the workflow diagram and the console schematics of section 4.
<code>docs/SPEC-umwl-tui.md</code> , <code>CLAUDE.md</code>	The contract of the application (state machine, UI, engine and workloader contract, security requirements, backlog) and the build, test and release notes for whoever extends it.
<code>legacy/umwl_loader.py</code>	The earlier plain-terminal loader (Python, standard library): the same ten steps, kept as fallback.
<code>legacy/reconcile_umwl.py</code>	Minimal, non-interactive IP reconciliation: from a <code>wkld-export</code> and the proposed CSV it writes <code>-to-create.csv</code> , <code>-existing.csv</code> , <code>-conflicts.csv</code> and a text report. It never writes to the PCE.
<code>examples/*-umwl-import*.csv</code> , <code>examples/*-ipl-import*.csv</code>	CSVs from a proof of concept, one row per IP; the <code>-v2</code> files are the format references (prefixed labels, name = role + IP, empty hostname, <code>eth0:<ip></code>). Templates only: they contain that lab's IPs.
<code>.gitignore</code>	Excludes the binaries, <code>pce.yaml</code> , <code>anon-map.json</code> , <code>*.anon.csv</code> , <code>*.real.csv</code> , logs, <code>runs/</code> , <code>dist/</code> and zips.

SOURCES — REPOSITORY

1. [illumio-workloader-import-kit](#) — [README](#), [CLAUDE.md](#), [docs/SPEC-umwl-tui.md](#), [.gitignore](#) — github.com/roschereric/illumio-workloader-import-kit

ABOUT THIS DOCUMENT

This guide was prepared by Eric Roscher – Illumio LATAM Pre-Sales Engineering to explain the end-to-end use of `umwl-tui` and the `illumio-workloader-import-kit` repository: exporting flows from the PCE, anonymizing them, running the analysis with Claude, reviewing the proposals and loading unmanaged workloads and IP lists with workloader. It is a working aid for the recipient and is not an official publication of Illumio, Inc.; it does not modify or replace official product documentation, contractual agreements, or statements of work. Validate all technical details against the official documentation for your specific product versions before acting on them. Illumio and the Illumio logo are trademarks of Illumio, Inc., used here to identify the subject matter.